

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №7
по дисциплине
«Программирование»
«Создание интерактивного дашборда»

Выполнила студентка группы ИВТб-1304-06-00 _____/Забелина А.А.

Проверил ст. преподаватель кафедры ЭВМ _____/Крутиков А.К.

Киров 2026

Цель работы

Освоение практических навыков создания интерактивных дашбордов в Python, включая:

- интеграцию pandas DataFrame с графическими библиотеками matplotlib и seaborn;
- встраивание графиков в окно Tkinter через бэкенд FigureCanvasTkAgg;
- реализацию событийно-ориентированной архитектуры;
- динамическую фильтрацию, агрегацию и feature engineering «на лету»;
- проектирование устойчивого процедурного кода без использования классов.

Описание задания

ВАРИАНТ 14: Геологические скважины

=====

Описание: Телеметрия нефтегазовых скважин: давление, температура, вибрация, состав флюида. Анализ помогает прогнозировать риски добычи.

Поля:

- ts (int32): время замера
- well_id (int16): ID скважины
- press (float32): пластовое давление, атм
- temp (float32): температура породы, °C
- vib (float32): уровень вибрации бура, мм/с
- fluid (float32): доля нефти в потоке, %

Очистка: press < 0. Заменить fluid > 100 на 100. Clip vib по 95-му перцентилю.

Пояснение: Давление не может быть отрицательным. Доля нефти >100% невозможна → ограничьте. Вибрацию ограничьте 95-м перцентилем, отсеив аварийные толчки.

Рисунок 1 — Вариант 14 (Описание предметной области)

1 Ход выполнения

В соответствии с вариантом выполнена фильтрация: удалены некорректные записи (Inf и NaN в числовых полях, отрицательные значения в поле press), доля нефти ограничена сверху 100%, вибрация ограничена 95-м перцентилем.

Произведена обрезка выбросов по скважинам (группировка по `well_id`, метрика `iqr`). Выбросы заменены на медиану группы.

Для построения графиков, необходимо добавить несколько производных признаков (категорийных полей, по которым удобно визуализировать данные).

Добавлено 4 поля:

- `data` - поле `ts` преобразовано из секунд в привычную для восприятия дату при помощи `pd.to_datetime`;
- `season` - на основе даты определяется время года (зима, весна, лето, осень);
- `v_oil` - условный объем нефти (используется для различного типа агрегации), рассчитывается как

$$press * \frac{fluid}{100} * (1 - \frac{vib}{100}),$$

отрицательные значения заменены на 0;

- `risk` - категориальное поле, требует ли скважина внимания. Если вибрация < 33-го перцентиля, то уровень риска `low`, от 33 до 66 - `medium`, выше 66 - `high`.

Категорийные поля по умолчанию имеют тип `object`, так как считаются строкой. Для оптимизации полям `season` и `risk` явно задан тип `category`.

2 Экранные формы

Каждый график можно сохранить в формат `.png` или `.pdf` при помощи кнопки Экспорт.

1. Линейный график

Для линейного графика доступен выбор скважины, по которой отображается график, тип агрегации (сумма, среднее или медиана), а также изменение масштаба отображения значений (день, месяц, год)

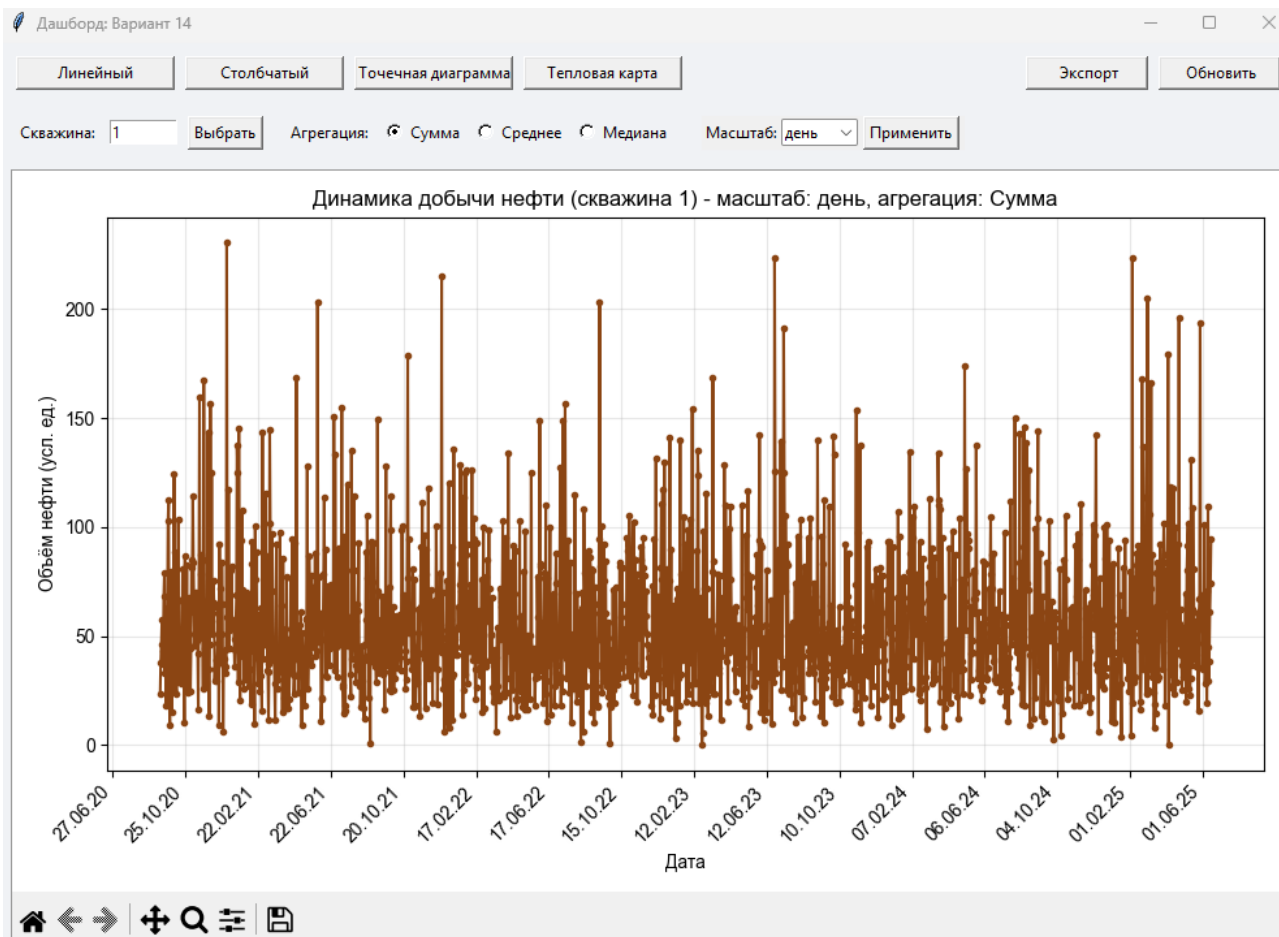


Рисунок 2 — Линейный график по дням

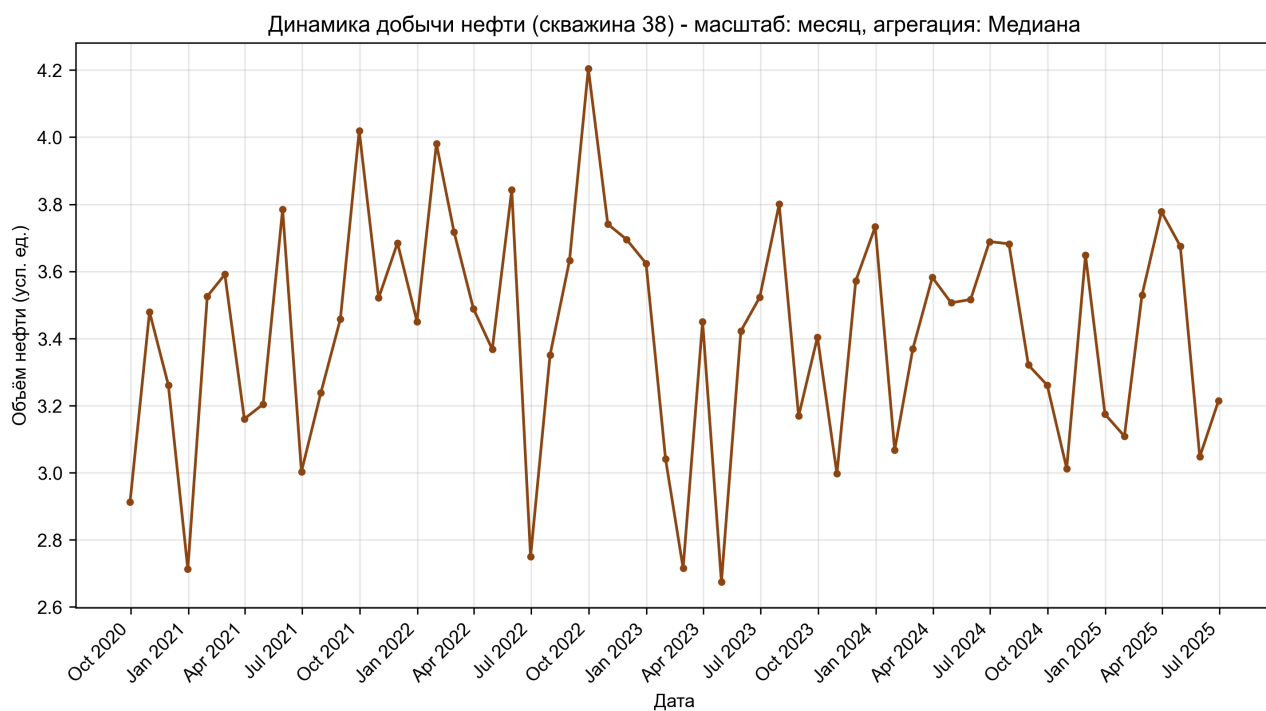


Рисунок 3 — Линейный график по месяцам

2. Столбчатая диаграмма

Для столбчатой диаграммы также доступен выбор скважины и типа агрегации, а также есть возможность выбора поля группировки (season или risk)

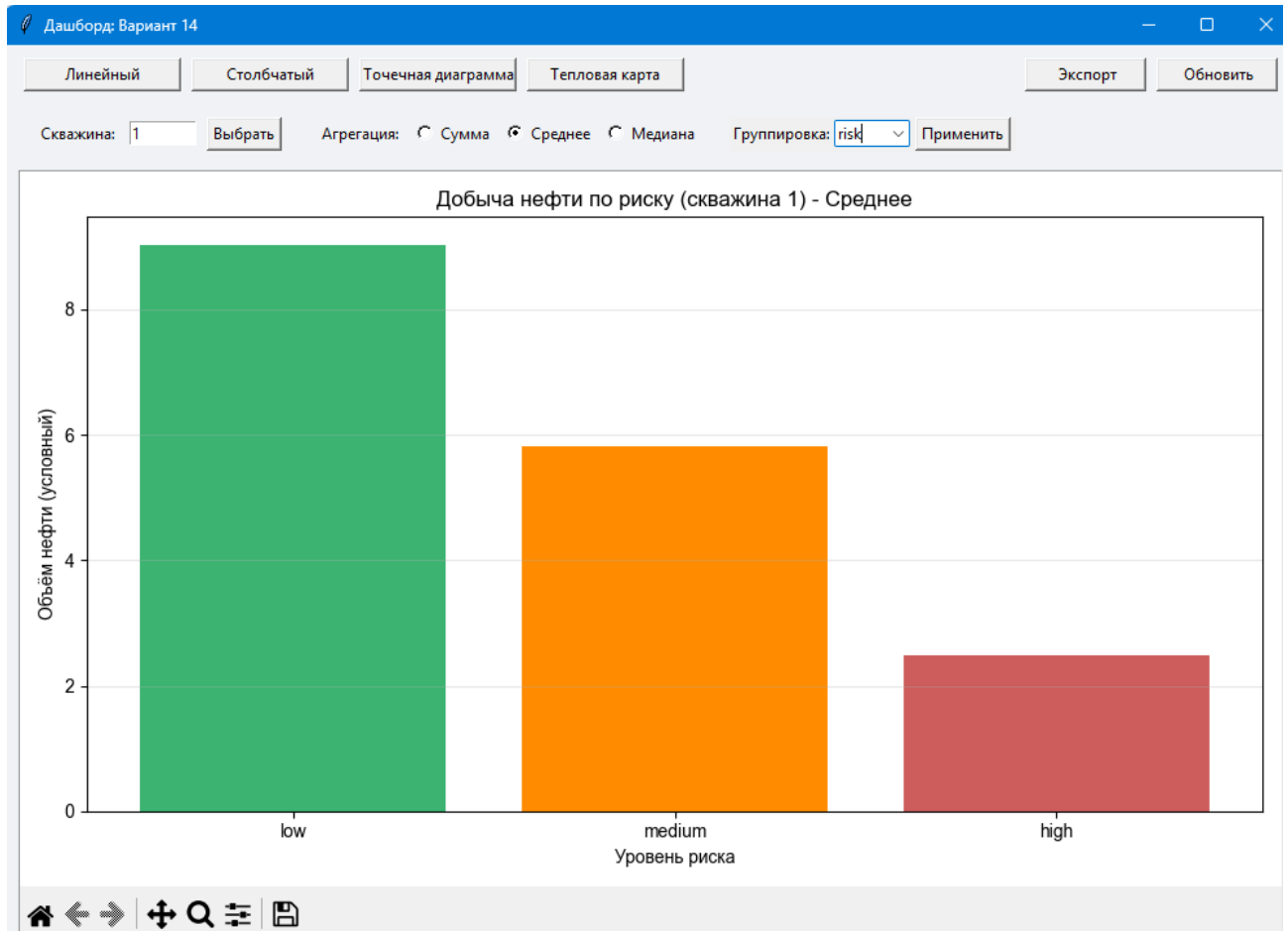


Рисунок 4 — Столбчатая диаграмма с группировкой по risk

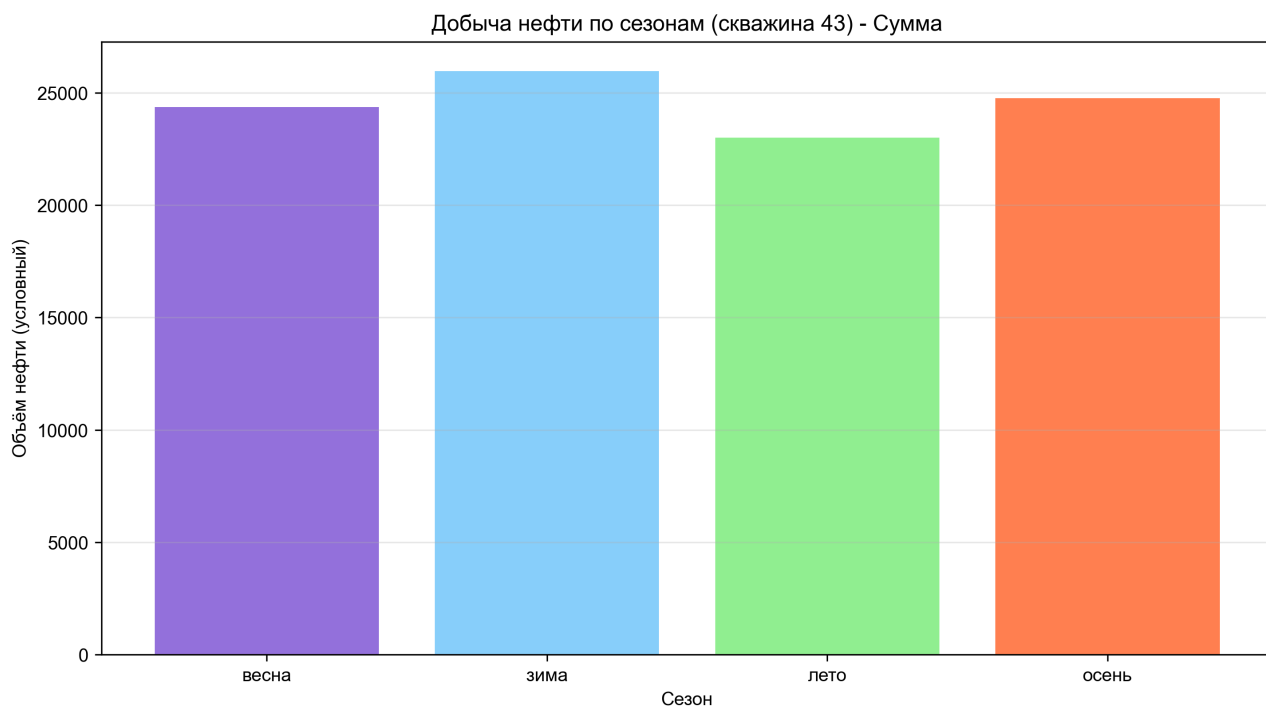


Рисунок 5 — Столбчатая диаграмма с группировкой по season

3. Точечная диаграмма

Точечная диаграмма позволяет пользователю выбрать скважину, а также поля, по которым будет построен график (ось X и ось Y). Можно выбрать цвет, которым будут раскрашены точки (risk или season), это позволит узнать зависят ли показатели от выбранных полей.

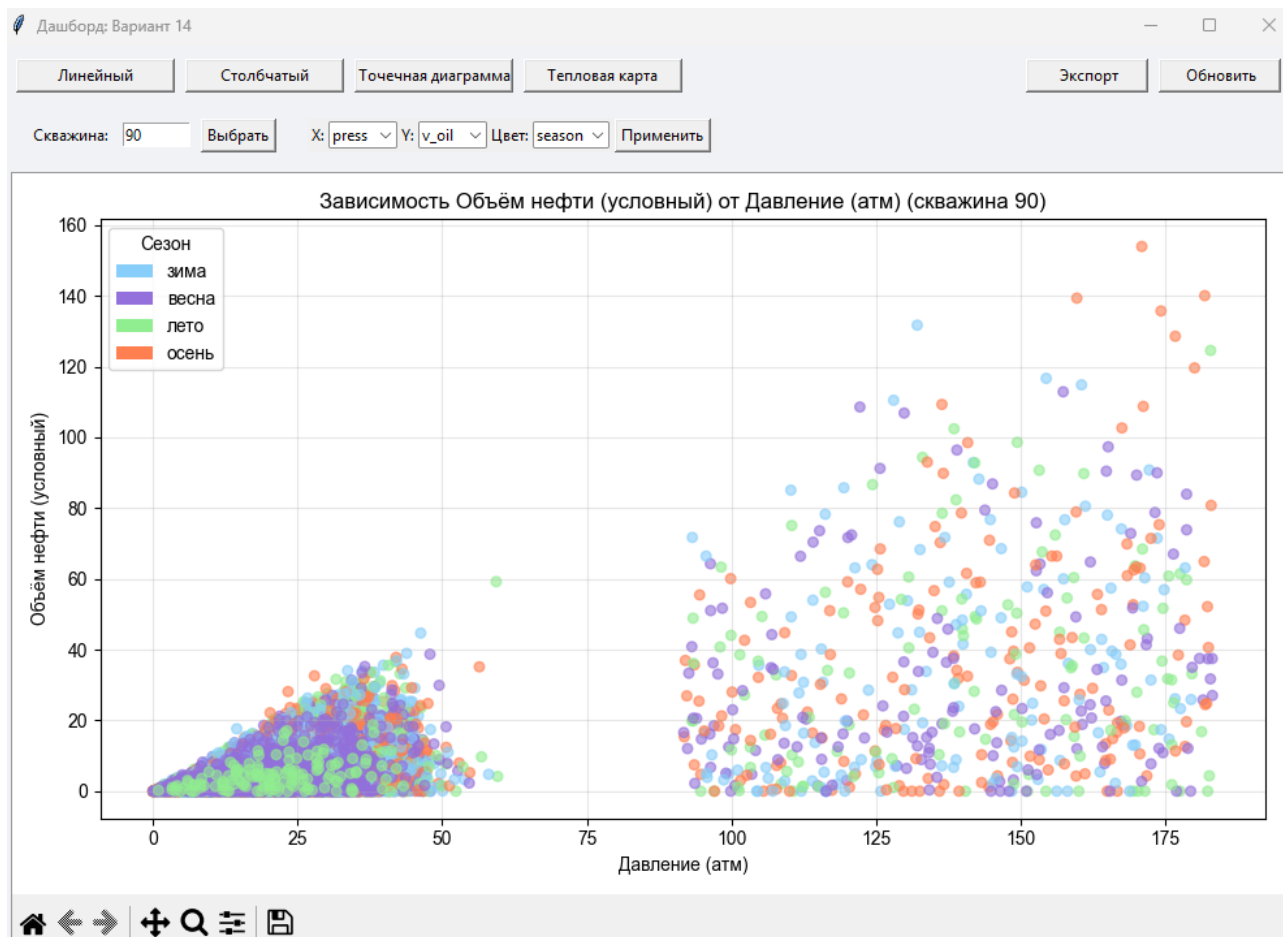


Рисунок 6 — Точечная диаграмма press к v_oil

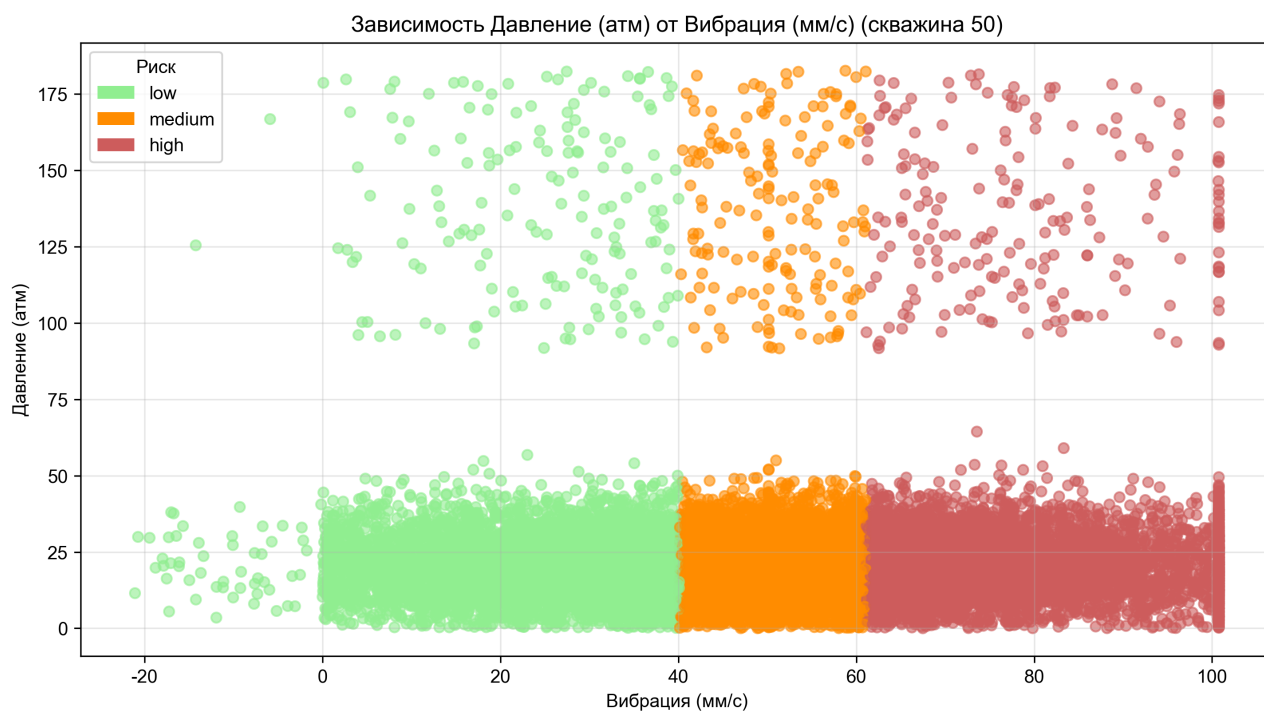


Рисунок 7 — Точечная диаграмма vib к press

4. Тепловая карта

В тепловой карте пользователь может выбрать тип агрегации и параметр, по которому будет построена тепловая карта. Данные разделены по временам года, что позволит выяснить зависимость значений от сезона.

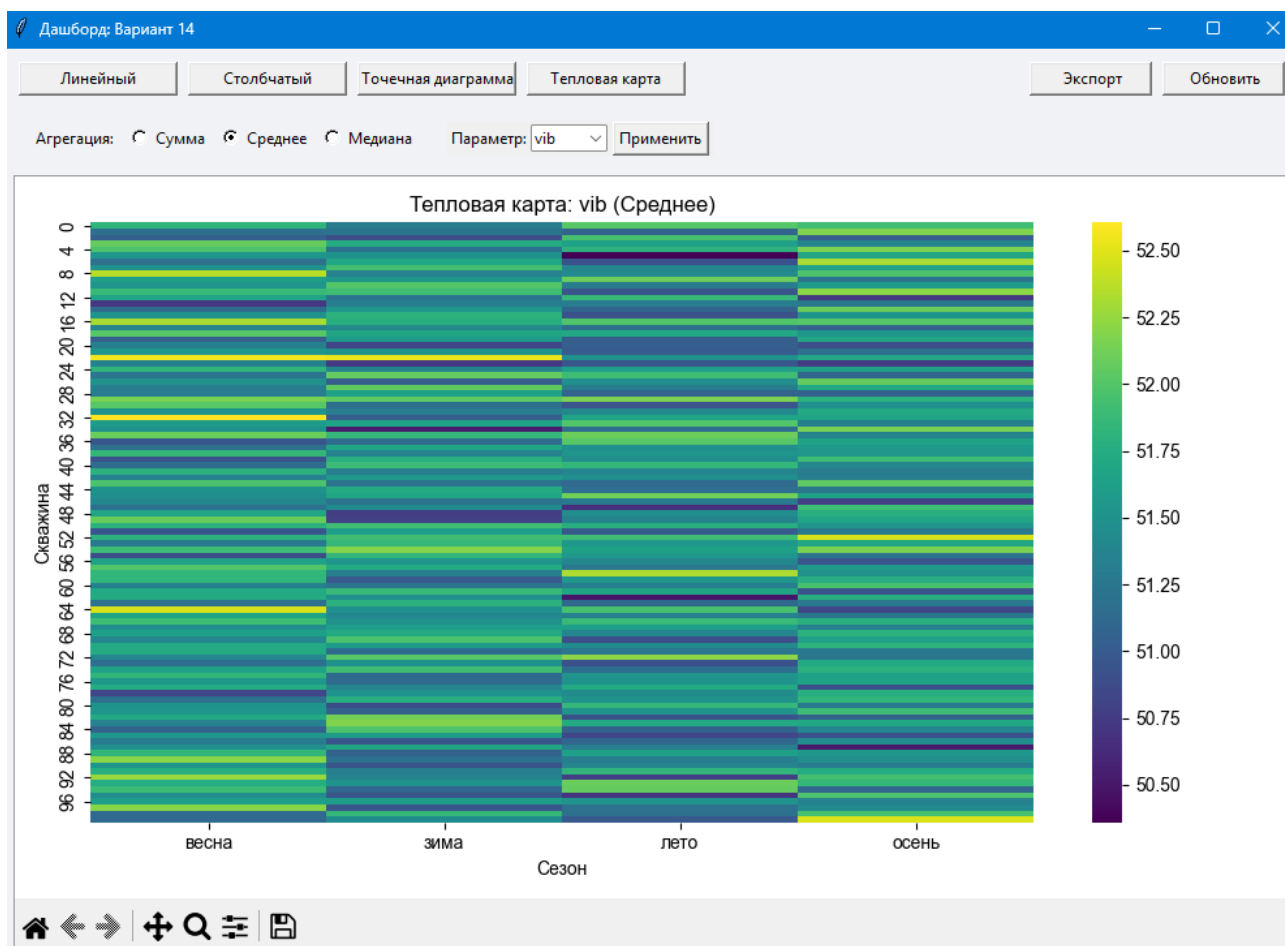


Рисунок 8 — Тепловая карта средней вибрации

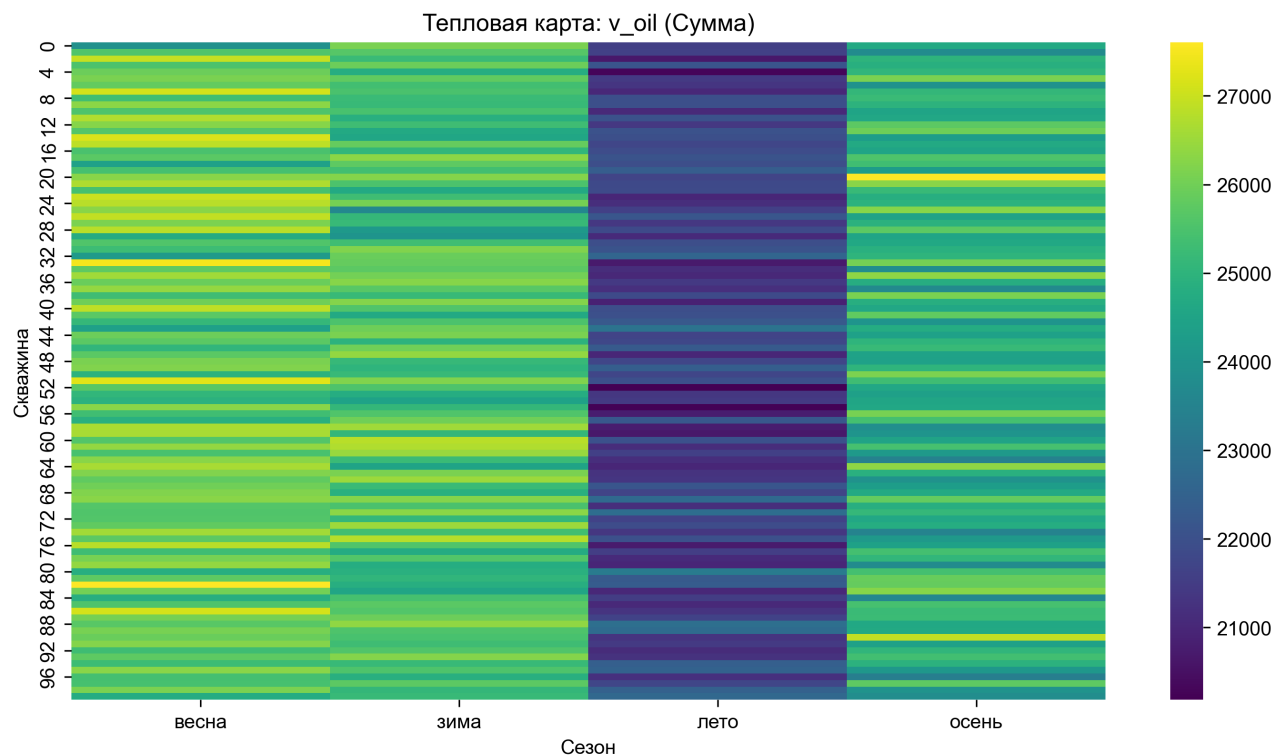


Рисунок 9 — Тепловая карта суммы объема нефти

3 Исходный код

```

1 import tkinter as tk
2 from tkinter import ttk, messagebox, filedialog
3 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg,
  ↳ NavigationToolbar2Tk
4 import matplotlib.pyplot as plt
5 import matplotlib.dates as mdates
6 from matplotlib.patches import Patch
7 import seaborn as sns
8 import pandas as pd
9 import numpy as np
10
11 # ----- Глобальные переменные -----
12 VARIANT_NUMBER = 14
13 CURRENT_ID = 1
14 CURRENT_SCALE = "день"
15 CURRENT_AGG = "sum"
16 HEAT_VALUE = "v_oil"
17 BAR_GROUP = "season"
18 SCATTER_X = "press"
19 SCATTER_Y = "v_oil"
20 SCATTER_COLOR = "risk"
21
22 # ----- Вспомогательные функции -----
23 # Распределение по временам года
24 def get_season(month):
25     if month in [12, 1, 2]:
26         return 'зима'
27     elif month in [3, 4, 5]:
28         return 'весна'
29     elif month in [6, 7, 8]:
30         return 'лето'

```

```

31     else:
32         return 'осень'
33
34 # ----- метрики IQR -----
35 def replace_outliers(group):
36     q1 = group.quantile(0.25)
37     q3 = group.quantile(0.75)
38     iqr = q3 - q1
39     lower = q1 - 1.5 * iqr
40     upper = q3 + 1.5 * iqr
41     median = group.median()
42
43     mask = (group < lower) | (group > upper)
44     group[mask] = median
45     return group
46
47 # ----- обработчики кнопок -----
48 # изменение текущей скважины
49 def set_well_id():
50     global CURRENT_ID
51     value = well_entry.get()
52
53     if not value.isdigit():
54         messagebox.showerror("Ошибка", "Введите число")
55         return
56
57     well_num = int(value)
58
59     if well_num not in df_work['well_id'].values:
60         messagebox.showerror("Ошибка", f"Скважины {well_num} не существует")
61         return
62
63     CURRENT_ID = well_num
64     print(f"Текущая скважина: {CURRENT_ID}")
65     refresh_data()
66
67 # агрегация в зависимости от масштаба (день / месяц / год)
68 def aggregate_by_scale(df, scale, agg_method):
69     df = df.copy()
70     df = df.set_index('date')
71
72     if agg_method == "sum":
73         df = df.resample(scale).sum()
74     elif agg_method == "mean":
75         df = df.resample(scale).mean()
76     elif agg_method == "median":
77         df = df.resample(scale).median()
78
79     return df.reset_index()
80
81 # масштаб (день / месяц / год)
82 def set_scale():
83     global CURRENT_SCALE
84     CURRENT_SCALE = scale_combo.get()
85     refresh_data()
86
87 # изменение типа агрегации (сумма / среднее / медиана)
88 def set_aggregation():
89     global CURRENT_AGG
90     CURRENT_AGG = agg_var.get()
91     refresh_data()
92
93 # Обработчик кнопки для тепловой карты
94 def set_heat_param():
95     global HEAT_VALUE

```

```

96     HEAT_VALUE = heat_value_combo.get()
97     if current_chart == "heat_map":
98         plot_heat_map()
99
100     # Обработчик кнопки для столбчатого графика
101     def set_bar_group():
102         global BAR_GROUP
103         BAR_GROUP = bar_group_combo.get()
104         if current_chart == "bar":
105             plot_bar()
106
107     # Обработчик кнопки для точечной диаграммы
108     def set_scatter_params():
109         global SCATTER_X, SCATTER_Y, SCATTER_COLOR
110         SCATTER_X = scatter_x_combo.get()
111         SCATTER_Y = scatter_y_combo.get()
112         SCATTER_COLOR = scatter_color_combo.get()
113         if current_chart == "scatter":
114             plot_scatter()
115
116     # ----- ПРЕДОБРАБОТКА ДАННЫХ (фильтрация, обрезка, вычисление производных
117     ↪ признаков, оптимизация) -----
117     def preprocess_data():
118         df_work = df_raw.copy()
119         # ----- 1. Фильтрация по условию варианта -----
120         float_fields = ['press', 'temp', 'vib', 'fluid']
121
122         mask_ni = np.zeros(df_work.shape[0], dtype=bool)
123         for field in float_fields:
124             mask_ni = mask_ni | np.isnan(df_work[field]) |
125                 ↪ np.isinf(df_work[field])
126
127         df_work = df_work[~mask_ni]
128         df_work = df_work[df_work['press'] >= 0]
129         df_work['fluid'] = np.clip(df_work['fluid'], None, 100)
130         vib_95 = np.percentile(df_work['vib'], 95)
131         df_work['vib'] = np.clip(df_work['vib'], None, vib_95)
132
133         # ----- 2. Обрезка выбросов (IQR по группам)
134         ↪ -----
135         df_work['vib'] =
136         ↪ df_work.groupby('well_id')['vib'].transform(replace_outliers)
137
138         # ----- 3. Безопасное вычисление производного признака -----
139         # Перевод из секунд в дату
140         df_work['date'] = pd.to_datetime(df_work['ts'], unit='s')
141
142         # Время года
143         df_work['season'] = df_work['date'].dt.month.apply(get_season)
144
145         # Условный объем нефти (для различного типа агрегации (сумма / среднее /
146         ↪ медиана)
147         df_work['v_oil'] = (df_work['press'] * (df_work['fluid'] / 100) * (1 -
148             ↪ df_work['vib'] / 100)).clip(lower=0)
149
150         # Требуется ли скважина внимания (уровень вибрации <33 процентиля low, >66
151         ↪ high, иначе medium)
152         p_33 = np.percentile(df_work['vib'], 33)
153         p_66 = np.percentile(df_work['vib'], 66)
154         df_work['risk'] = 'medium'
155         df_work.loc[df_work['vib'] < p_33, 'risk'] = 'low'
156         df_work.loc[df_work['vib'] > p_66, 'risk'] = 'high'
157
158         # ----- 4. Оптимизация категориальных полей -----

```

```

153     # Перевод категориальных полей из типа object в category для оптимизации
154     df_work['season'] = df_work['season'].astype('category')
155     df_work['risk'] = df_work['risk'].astype('category')
156
157     return df_work
158
159     #----- ФУНКЦИИ ОТРИСОВКИ ГРАФИКОВ -----
160     #Линейный график
161     def plot_line():
162         global current_chart
163         current_chart = "line"
164         well_frame.pack(side=tk.LEFT, padx=10)
165         agg_frame.pack(side=tk.LEFT, padx=10)
166         scale_frame.pack(side=tk.LEFT, padx=10)
167         bar_frame.pack_forget()
168         scatter_frame.pack_forget()
169         heat_frame.pack_forget()
170
171         fig.clear()
172         ax = fig.add_subplot(111)
173
174         df_one_well = df_work.loc[df_work['well_id'] == CURRENT_ID, ['date',
175             ↪ 'v_oil']].copy()
176         df_one_well = df_one_well.sort_values('date')
177
178         if CURRENT_SCALE == "день":
179             rule = 'D'
180         elif CURRENT_SCALE == "месяц":
181             rule = 'ME'
182         else:
183             rule = 'YE' # год
184
185         df_agg = aggregate_by_scale(df_one_well, rule, CURRENT_AGG)
186
187         ax.plot(df_agg['date'], df_agg['v_oil'], linewidth=1.5,
188             ↪ color='saddlebrown', marker='o', markersize=3)
189
190         agg_names = {"sum": "Сумма", "mean": "Среднее", "median": "Медиана"}
191         ax.set_title(
192             f'Динамика добычи нефти (скважина {CURRENT_ID}) - масштаб:
193             ↪ {CURRENT_SCALE}, агрегация: {agg_names[CURRENT_AGG]}')
194         ax.set_xlabel('Дата')
195         ax.set_ylabel('Объём нефти (усл. ед.)')
196         ax.grid(True, alpha=0.3)
197
198         if CURRENT_SCALE == "день":
199             ax.xaxis.set_major_formatter(mdates.DateFormatter('%d.%m.%y'))
200             ax.xaxis.set_major_locator(mdates.DayLocator(interval=120))
201         elif CURRENT_SCALE == "месяц":
202             ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
203             ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
204         elif CURRENT_SCALE == "год":
205             ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
206             ax.xaxis.set_major_locator(mdates.YearLocator())
207
208         plt.setp(ax.xaxis.get_majorticklabels(), rotation=45, ha='right')
209         fig.tight_layout()
210         canvas.draw_idle()
211
212     #Столбчатая диаграмма
213     def plot_bar():
214         global current_chart
215         current_chart = "bar"
216         well_frame.pack(side=tk.LEFT, padx=10)

```

```

214     agg_frame.pack(side=tk.LEFT, padx=10)
215     bar_frame.pack(side=tk.LEFT, padx=10)
216     scale_frame.pack_forget()
217     scatter_frame.pack_forget()
218     heat_frame.pack_forget()
219
220     fig.clear()
221     ax = fig.add_subplot(111)
222
223     if BAR_GROUP == "season":
224         grouped = df_work[df_work['well_id'] ==
225             ↪ CURRENT_ID].groupby('season')['v_oil'].agg(CURRENT_AGG)
226         x_labels = grouped.index.tolist()
227         values = grouped.values
228         x_label = "Сезон"
229         title_group = "сезонам"
230     else: # risk
231         grouped = df_work[df_work['well_id'] ==
232             ↪ CURRENT_ID].groupby('risk')['v_oil'].agg(CURRENT_AGG)
233         order = ['low', 'medium', 'high']
234         grouped = grouped.reindex(order)
235         x_labels = grouped.index.tolist()
236         values = grouped.values
237         x_label = "Уровень риска"
238         title_group = "риску"
239
240     agg_names = {"sum": "Сумма", "mean": "Среднее", "median": "Медиана"}
241
242     ax.bar(x_labels, values, color=['mediumseagreen', 'darkorange',
243         ↪ 'indianred'] if BAR_GROUP == "risk" else ['mediumpurple',
244         ↪ 'lightskyblue', 'lightgreen', 'coral'])
245
246     ax.set_title(f'Добыча нефти по {title_group} (скважина {CURRENT_ID}) -
247         ↪ {agg_names[CURRENT_AGG]}')
248     ax.set_xlabel(x_label)
249     ax.set_ylabel('Объём нефти (условный)')
250     ax.grid(True, alpha=0.3, axis='y')
251
252     fig.tight_layout()
253     canvas.draw_idle()
254
255     #Точечная диаграмма
256     def plot_scatter():
257         global current_chart
258         current_chart = "scatter"
259         well_frame.pack(side=tk.LEFT, padx=10)
260         agg_frame.pack_forget()
261         scatter_frame.pack(side=tk.LEFT, padx=10)
262         scale_frame.pack_forget()
263         bar_frame.pack_forget()
264         heat_frame.pack_forget()
265
266         fig.clear()
267         ax = fig.add_subplot(111)
268
269         df_one_well = df_work[df_work['well_id'] == CURRENT_ID]
270
271         if SCATTER_COLOR == "risk":
272             colors = {'low': 'lightgreen', 'medium': 'darkorange', 'high':
273                 ↪ 'indianred'}
274             point_colors = df_one_well['risk'].map(colors)
275             legend_title = "Риск"
276             legend_elements = [
277                 Patch(facecolor='lightgreen', label='low'),

```

```

272         Patch(facecolor='darkorange', label='medium'),
273         Patch(facecolor='indianred', label='high')
274     ]
275     else: # season
276         season_colors = {'зима': 'lightskyblue', 'весна': 'mediumpurple',
277             ↪ 'лето': 'lightgreen', 'осень': 'coral'}
278         point_colors = df_one_well['season'].map(season_colors)
279         legend_title = "Сезон"
280         legend_elements = [
281             Patch(facecolor='lightskyblue', label='зима'),
282             Patch(facecolor='mediumpurple', label='весна'),
283             Patch(facecolor='lightgreen', label='лето'),
284             Patch(facecolor='coral', label='осень')
285         ]
286     ax.scatter(
287         df_one_well[SCATTER_X],
288         df_one_well[SCATTER_Y],
289         c=point_colors,
290         alpha=0.6,
291         s=30
292     )
293
294     names = {"press": "Давление (атм)", "temp": "Температура (°C)",
295             ↪ "vib": "Вибрация (мм/с)", "v_oil": "Объем нефти (условный)"}
296
297     ax.set_xlabel(names.get(SCATTER_X, SCATTER_X))
298     ax.set_ylabel(names.get(SCATTER_Y, SCATTER_Y))
299     ax.set_title(f'Зависимость {names[SCATTER_Y]} от {names[SCATTER_X]}
300             ↪ (скважина {CURRENT_ID})')
301     ax.grid(True, alpha=0.3)
302
303     ax.legend(handles=legend_elements, title=legend_title)
304
305     fig.tight_layout()
306     canvas.draw_idle()
307
308     #Тепловая карта
309     def plot_heat_map():
310         global current_chart
311         current_chart = "heat_map"
312         agg_frame.pack(side=tk.LEFT, padx=10)
313         heat_frame.pack(side=tk.LEFT, padx=10)
314         scale_frame.pack_forget()
315         bar_frame.pack_forget()
316         scatter_frame.pack_forget()
317         well_frame.pack_forget()
318
319         fig.clear()
320         ax = fig.add_subplot(111)
321
322         pivot_data = df_work.pivot_table(
323             values=HEAT_VALUE,
324             index='well_id',
325             columns='season',
326             aggfunc=CURRENT_AGG
327         )
328
329         sns.heatmap(pivot_data, annot=False, cmap='viridis', ax=ax)
330
331         agg_names = {"sum": "Сумма", "mean": "Среднее", "median": "Медиана"}
332         ax.set_title(f'Тепловая карта: {HEAT_VALUE} ({agg_names[CURRENT_AGG]})')
333         ax.set_xlabel('Сезон')
334         ax.set_ylabel('Скважина')

```

```

334     fig.tight_layout()
335     canvas.draw_idle()
336
337
338     #Обновление данных
339     def refresh_data():
340         global df_work
341         df_work = preprocess_data()
342         if current_chart == "line":
343             plot_line()
344         elif current_chart == "bar":
345             plot_bar()
346         elif current_chart == "scatter":
347             plot_scatter()
348         elif current_chart == "heat_map":
349             plot_heat_map()
350
351     #Экспорт в .png и .pdf
352     def export_plot():
353         filepath = filedialog.asksaveasfilename(defaultextension=".png",
354                                                 filetype=[("PNG", "*.png"),
355                                                         ↪ ("PDF", "*.pdf")])
356
357         if filepath:
358             fig.savefig(filepath, dpi=300, bbox_inches='tight')
359
360
361     # ----- ОСНОВНАЯ ОБЛАСТЬ (ИНТЕРФЕЙС) -----
362     ↪ -----
363     df_raw = None # Исходные данные
364     df_work = None # Рабочая копия
365     fig = plt.Figure(figsize=(9, 5.5), dpi=100)
366     canvas = None
367     current_chart = "line"
368     # ----- Настройка шрифтов для кириллицы -----
369     plt.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans', 'Liberation Sans']
370     plt.rcParams['axes.unicode_minus'] = False
371     # Загрузка и диагностика
372     df_raw = pd.read_csv('data.csv')
373
374     root = tk.Tk()
375     root.title(f"Дашборд: Вариант {VARIANT_NUMBER}")
376     root.geometry("1000x700")
377     root.configure(bg="#f0f2f5")
378     # ----- Контейнер для кнопок -----
379     ctrl_frame = tk.Frame(root, bg="#f0f2f5")
380     ctrl_frame.pack(side=tk.TOP, fill=tk.X, padx=10, pady=10)
381     # ----- Контейнер для параметров -----
382     par_frame = tk.Frame(root, bg="#f0f2f5")
383     par_frame.pack(side=tk.TOP, fill=tk.X, padx=10, pady=10)
384     # ----- Контейнер для графика -----
385     plot_frame = tk.Frame(root, bg="white", relief=tk.SUNKEN, bd=1)
386     plot_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
387     # ----- Адаптер matplotlib -----
388     canvas = FigureCanvasTkAgg(fig, master=plot_frame)
389     canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
390     # ----- Панель инструментов -----
391     toolbar = NavigationToolbar2Tk(canvas, plot_frame)
392     toolbar.update()
393     toolbar.pack(side=tk.TOP, fill=tk.X)
394
395     # ----- Панель кнопок -----
396     tk.Button(ctrl_frame, text="Линейный", command=plot_line,
397             ↪ width=16).pack(side=tk.LEFT, padx=4)
398     tk.Button(ctrl_frame, text="Столбчатый", command=plot_bar,
399             ↪ width=16).pack(side=tk.LEFT, padx=4)

```



```

394 tk.Button(ctrl_frame, text="Точечная диаграмма", command=plot_scatter,
    ↳ width=16).pack(side=tk.LEFT, padx=4)
395 tk.Button(ctrl_frame, text="Тепловая карта", command=plot_heat_map,
    ↳ width=16).pack(side=tk.LEFT, padx=4)
396
397 tk.Button(ctrl_frame, text="Обновить", command=refresh_data,
    ↳ width=12).pack(side=tk.RIGHT, padx=4)
398 tk.Button(ctrl_frame, text="Экспорт", command=export_plot,
    ↳ width=12).pack(side=tk.RIGHT, padx=4)
399
400 for widget in par_frame.winfo_children():
401     widget.destroy()
402
403 #----- Выбор скважины -----
404 well_frame = tk.Frame(par_frame, bg="#f0f2f5")
405 tk.Label(well_frame, text="Скважина:", bg="#f0f2f5").pack(side=tk.LEFT,
    ↳ padx=4)
406 well_entry = tk.Entry(well_frame, width=8)
407 well_entry.pack(side=tk.LEFT, padx=4)
408 well_entry.insert(0, str(CURRENT_ID))
409 tk.Button(well_frame, text="Выбрать", command=set_well_id).pack(side=tk.LEFT,
    ↳ padx=4)
410 well_frame.pack(side=tk.LEFT, padx=10)
411
412 #----- Агрегация -----
413 agg_frame = tk.Frame(par_frame, bg="#f0f2f5")
414 tk.Label(agg_frame, text="Агрегация:", bg="#f0f2f5").pack(side=tk.LEFT,
    ↳ padx=4)
415 agg_var = tk.StringVar(value="sum")
416 tk.Radiobutton(agg_frame, text="Сумма", variable=agg_var, value="sum",
    ↳ command=set_aggregation, bg="#f0f2f5").pack(
417     side=tk.LEFT, padx=2)
418 tk.Radiobutton(agg_frame, text="Среднее", variable=agg_var, value="mean",
    ↳ command=set_aggregation, bg="#f0f2f5").pack(
419     side=tk.LEFT, padx=2)
420 tk.Radiobutton(agg_frame, text="Медиана", variable=agg_var, value="median",
    ↳ command=set_aggregation, bg="#f0f2f5").pack(
421     side=tk.LEFT, padx=2)
422 agg_frame.pack(side=tk.LEFT, padx=10)
423
424 #----- Масштаб (день / месяц / год) только для линейного графика
    ↳ -----
425 scale_frame = tk.Frame(par_frame)
426 tk.Label(scale_frame, text="Масштаб:").pack(side=tk.LEFT)
427 scale_combo = ttk.Combobox(scale_frame, values=["день", "месяц", "год"],
    ↳ width=6)
428 scale_combo.set("день")
429 scale_combo.pack(side=tk.LEFT)
430 tk.Button(scale_frame, text="Применить", command=set_scale).pack(side=tk.LEFT,
    ↳ padx=4)
431
432 #----- Группировка только для столбчатой диаграммы
    ↳ -----
433 bar_frame = tk.Frame(par_frame)
434 tk.Label(bar_frame, text="Группировка:").pack(side=tk.LEFT)
435 bar_group_combo = ttk.Combobox(bar_frame, values=["season", "risk"], width=6)
436 bar_group_combo.set("season")
437 bar_group_combo.pack(side=tk.LEFT)
438 tk.Button(bar_frame, text="Применить",
    ↳ command=set_bar_group).pack(side=tk.LEFT, padx=4)
439
440 #----- Выбор осей (только для точечной диаграммы)
    ↳ -----

```



```

441 scatter_frame = tk.Frame(par_frame)
442 tk.Label(scatter_frame, text="X:").pack(side=tk.LEFT)
443 scatter_x_combo = ttk.Combobox(scatter_frame, values=["press", "temp", "vib",
444     ↪ "v_oil"], width=5)
444 scatter_x_combo.set("press")
445 scatter_x_combo.pack(side=tk.LEFT)
446 tk.Label(scatter_frame, text="Y:").pack(side=tk.LEFT)
447 scatter_y_combo = ttk.Combobox(scatter_frame, values=["v_oil", "press",
448     ↪ "temp", "vib"], width=5)
448 scatter_y_combo.set("v_oil")
449 scatter_y_combo.pack(side=tk.LEFT)
450 tk.Label(scatter_frame, text="Цвет:").pack(side=tk.LEFT)
451 scatter_color_combo = ttk.Combobox(scatter_frame, values=["risk", "season"],
452     ↪ width=6)
452 scatter_color_combo.set("risk")
453 scatter_color_combo.pack(side=tk.LEFT)
454 tk.Button(scatter_frame, text="Применить",
455     ↪ command=set_scatter_params).pack(side=tk.LEFT, padx=4)
456
456 #----- Параметры для тепловой карты
457     ↪ -----
457 heat_frame = tk.Frame(par_frame)
458 tk.Label(heat_frame, text="Параметр:").pack(side=tk.LEFT)
459 heat_value_combo = ttk.Combobox(heat_frame, values=["v_oil", "press", "temp",
460     ↪ "vib"], width=6)
460 heat_value_combo.set("v_oil")
461 heat_value_combo.pack(side=tk.LEFT)
462 tk.Button(heat_frame, text="Применить",
463     ↪ command=set_heat_param).pack(side=tk.LEFT, padx=4)
464
464 scale_frame.pack_forget()
465 bar_frame.pack_forget()
466 scatter_frame.pack_forget()
467 heat_frame.pack_forget()
468
469 df_work = preprocess_data()
470 plot_line()
471 root.mainloop()

```

4 Анализ результатов

Представление таблиц в виде графиков очень удобно для анализа и поиска закономерностей. Например, при построении тепловой карты для объема добытой нефти, сразу можно заметить низкие значения для летнего сезона. Нахождение таких закономерностей очень важно для аналитики данных, без визуализации найти различные закономерности и зависимости полей было бы очень сложно.

Поскольку датасет сгенерирован случайным образом, то сложно выявить конкретные закономерности и зависимость параметров. Однако при использовании реальных данных, построение графиков будет являться мощ-

ным инструментом обработки данных.

Выводы

В ходе выполнения лабораторной работы была освоена работа с библиотеками pandas (для первичной обработки данных), matplotlib (для четкой настройки графиков), seaborn (для легкой и быстрой визуализации), а также Tkinter для графического интерфейса.

Выполнена предварительная обработка данных, удалены некорректные значения, аномалии, добавлены категориальные поля по которым удобно визуализировать данные.

В результате разработан дашборд, который позволяет визуализировать данные, строя 4 типа графиков (линейный, столбчатый, точечный, тепловая карта). Для каждого графика доступно динамическое изменение параметров, а также сохранение в формате .png и .pdf.

Построение графиков является очень мощным инструментом для анализа больших объемов данных. Графики позволяют быстро оценить общую картину: найти аномалии, которые было бы сложно выявить в таблице. Визуализация данных сокращает время анализа в десятки раз. Вместо изучения миллионов строк таблицы достаточно посмотреть на график, чтобы понять динамику процесса, сравнить показатели разных групп или обнаружить неочевидные зависимости между параметрами.

Ссылка на репозиторий GitHub: <https://github.com/umshik/Lab7>